



Department of Electrical and Information Engineering

University of Ruhuna

**Project Analysis Report
High Performance Computing
EE7218 / EC7207**

Parallel PageRank Algorithm for Social Media Influencer Detection

**using OpenMP, POSIX Threads, MPI, CUDA, and Hybrid
Programming**

Group Number: 09

EG/2021/4407	Arachchi K.W.A.R.L.W
EG/2021/4408	Arachchi
EG/2021/4410	Arachchi W.A.H.U.W

May 22, 2026

Contents

Abstract	2
1 Parallel Programming Concept Application	3
1.1 Project Overview	3
1.2 PageRank Algorithm	3
1.3 Graph Representation	4
1.4 Datasets	5
1.5 Serial Implementation	5
1.6 OpenMP Implementation	5
1.7 POSIX Threads Implementation	6
1.8 MPI Implementation	6
1.9 CUDA Implementation	7
1.10 Hybrid MPI + OpenMP Implementation	7
1.11 Hybrid MPI + CUDA Implementation	8
2 Accuracy Analysis	9
2.1 RMSE Definition	9
2.2 PageRank Sum Validation	9
2.3 Accuracy Results	10
2.4 Accuracy Discussion	10
3 Performance and Scalability Analysis	11
3.1 Experimental Environment	11
3.2 Speedup and Efficiency Metrics	11
3.3 MPI Performance Results	12
3.4 CUDA Block Size Experiment	12
3.5 CUDA Dataset Performance	12
3.6 Hybrid MPI + OpenMP Performance	13
3.7 Hybrid MPI + CUDA Performance	13
3.8 Performance Discussion	13
4 User Interface Demonstration	15
4.1 Flask Dashboard Overview	15
4.2 UI Implementation	15
5 Conclusion	17

Abstract

This project implements and analyzes the PageRank algorithm using multiple High Performance Computing (HPC) programming models. PageRank is an iterative graph algorithm used to estimate the relative importance of nodes in a directed graph. In this project, the algorithm is applied to real-world graph datasets such as social networks and web graphs in order to identify influential nodes.

The implementation begins with a serial CPU version, which is used as the baseline for both accuracy and performance comparison. The project then extends the computation using shared-memory programming with OpenMP and POSIX Threads, distributed-memory programming with MPI, GPU acceleration with CUDA, and two hybrid approaches: MPI + OpenMP and MPI + CUDA. The correctness of each implementation is verified by comparing its output with the serial output using Root Mean Square Error (RMSE). Performance is analyzed using execution time, speedup, efficiency, and scalability across different datasets and parallel configurations.

The experimental results show that all tested parallel and hybrid implementations produce PageRank values very close to the serial baseline. The PageRank sum remains approximately 1.000000, and RMSE values are close to zero. The CUDA implementation provides performance improvement for some larger datasets, while MPI and hybrid implementations show that speedup depends strongly on problem size, communication overhead, hardware availability, and graph structure. In particular, the MPI + CUDA implementation is designed for multi-GPU environments, but was tested on a single-GPU laptop, which limits its full performance potential.

Chapter 1

Parallel Programming Concept Application

1.1 Project Overview

The aim of this project is to parallelize the PageRank algorithm using different HPC programming paradigms and analyze their correctness and performance. PageRank is suitable for parallel computing because the rank value of each node can be updated independently during each iteration once the previous iteration's rank vector is available.

The project includes the following implementations:

- Serial PageRank implementation
- OpenMP shared-memory implementation
- POSIX Threads shared-memory implementation
- MPI distributed-memory implementation
- CUDA GPU implementation
- Hybrid MPI + OpenMP implementation
- Hybrid MPI + CUDA implementation

The same graph input datasets are used across implementations so that execution time, speedup, efficiency, and accuracy can be compared fairly.

1.2 PageRank Algorithm

PageRank assigns an importance score to each node in a directed graph. A node receives a higher PageRank value if many nodes point to it or if it receives links from other important nodes. Therefore, PageRank is not only a simple in-degree count; it also considers the importance of the nodes providing incoming links.

For a graph with N nodes, the PageRank of node i is calculated as:

$$PR(i) = \frac{1-d}{N} + d \left(\frac{S_{dangling}}{N} + \sum_{j \in In(i)} \frac{PR(j)}{OutDegree(j)} \right) \quad (1.1)$$

where:

- $PR(i)$ is the PageRank value of node i
- d is the damping factor, set to 0.85
- N is the total number of nodes
- $In(i)$ is the set of nodes that point to node i
- $OutDegree(j)$ is the number of outgoing edges from node j
- $S_{dangling}$ is the total PageRank contribution from dangling nodes

Dangling nodes are nodes with no outgoing edges. Their PageRank values are distributed equally among all nodes to prevent rank loss. The PageRank values are updated iteratively until the global change between two consecutive iterations becomes smaller than the tolerance value or the maximum number of iterations is reached.

In this project:

- Damping factor: 0.85
- Maximum iterations: 50
- Convergence tolerance: 10^{-6}

1.3 Graph Representation

The input graph is read from a text file where each line contains a directed edge:

source destination

For example, 0 1 means that node 0 has a directed edge to node 1.

The graph is stored using an incoming-edge structure similar to Compressed Sparse Column (CSC) representation. The main arrays are:

- **src**: stores source nodes of edges
- **dst**: stores destination nodes of edges
- **outDegree**: stores number of outgoing edges for each node
- **inEdges**: stores incoming source nodes for each destination node
- **inPtr**: stores pointer offsets for incoming edges of each node

This representation is efficient for PageRank because the rank of a node depends on its incoming edges.

1.4 Datasets

Several graph datasets were used to test correctness and performance. The datasets include a small sample graph for correctness testing and larger real-world datasets for performance analysis.

Dataset	Nodes	Edges	Description
Sample Graph	4	6	Small graph used for correctness testing.
soc-Epinions1	75,888	508,837	Directed who-trusts-whom social network graph.
web-Google	916,428	5,105,039	Web graph where nodes represent web pages and edges represent hyperlinks.
web-BerkStan	685,231	7,600,595	Berkeley-Stanford web graph dataset.
WikiTalk	2,394,385	5,021,410	Wikipedia user talk network dataset.

Table 1.1: Datasets used for PageRank evaluation

1.5 Serial Implementation

The serial implementation runs on a single CPU core and is used as the baseline. It initializes all PageRank values equally and repeatedly updates each node's rank using the PageRank formula. The serial version also calculates dangling-node contribution and convergence difference in a sequential manner.

The serial implementation is important because:

- It provides the reference output for accuracy comparison.
- It provides the baseline execution time for speedup calculation.
- It helps verify whether the parallel implementations produce correct results.

1.6 OpenMP Implementation

The OpenMP implementation uses shared-memory parallel programming. The main PageRank loops are parallelized using `#pragma omp parallel for`. OpenMP automatically manages thread creation, scheduling, and synchronization.

The following operations are parallelized:

- Initializing PageRank values
- Calculating dangling-node contribution using reduction
- Updating PageRank values for all nodes
- Computing convergence difference using reduction

OpenMP is suitable for multi-core CPUs because all threads share the same memory space. However, synchronization and reduction operations can introduce overhead.

1.7 POSIX Threads Implementation

The POSIX Threads implementation uses manual thread management. The total set of nodes is divided among threads, and each thread computes PageRank values for its assigned node range. Compared with OpenMP, Pthreads requires explicit thread creation, workload division, joining, and synchronization.

Key characteristics of the Pthreads implementation are:

- Explicit control over thread creation and node partitioning
- Shared-memory parallel execution
- Manual synchronization where shared values are updated
- More implementation complexity than OpenMP

Pthreads provides a lower-level understanding of shared-memory parallelism compared with OpenMP.

1.8 MPI Implementation

The MPI implementation uses distributed-memory programming. Multiple MPI processes are created, and the graph nodes are divided among those processes. Each process computes PageRank values only for its assigned node block.

The workflow is:

1. Process 0 reads the graph from the dataset.
2. Process 0 broadcasts the full graph data to all MPI processes using `MPI_Bcast`.
3. Nodes are divided among MPI processes.
4. Each process computes new PageRank values for its assigned nodes.
5. `MPI_Allreduce` is used to calculate global dangling-node contribution.
6. `MPI_Allreduce` is used to calculate global convergence difference.
7. `MPI_Allgatherv` is used to share updated PageRank values among all processes.
8. Process 0 prints and saves the final output.

MPI is useful for distributed systems and multi-node execution, but communication overhead can reduce speedup for small datasets or high process counts.

1.9 CUDA Implementation

The CUDA implementation uses GPU acceleration. The CPU loads the graph and copies the required arrays to GPU memory. CUDA kernels are launched so that GPU threads compute PageRank values in parallel.

The CUDA design uses:

- One GPU thread for one graph node
- CUDA kernels for dangling-node calculation
- CUDA kernels for PageRank update
- CUDA kernels for convergence difference calculation
- CUDA block sizes such as 128, 256, and 512

CUDA can provide high parallelism because thousands of GPU threads can execute concurrently. However, performance depends on dataset size, memory transfer overhead, atomic operations, GPU occupancy, and graph memory access patterns.

1.10 Hybrid MPI + OpenMP Implementation

The first hybrid implementation combines MPI and OpenMP. MPI is used to divide the graph among processes, while OpenMP is used inside each MPI process to parallelize the computation over CPU threads.

The model is:

$$\text{MPI processes} \times \text{OpenMP threads per process}$$

For example:

$$2 \text{ MPI processes} \times 2 \text{ OpenMP threads} = 4 \text{ total workers}$$

The workflow is:

1. MPI distributes node ranges among processes.
2. Each MPI process uses OpenMP threads to compute PageRank for its local node range.
3. MPI collective operations combine global dangling sum and convergence difference.
4. MPI shares updated PageRank blocks after each iteration.

This hybrid method combines distributed-memory and shared-memory parallelism. It is suitable for systems where each node has multiple CPU cores.

1.11 Hybrid MPI + CUDA Implementation

The second hybrid implementation combines MPI and CUDA. MPI distributes node partitions among processes, while CUDA performs the PageRank computation on the GPU for each process's assigned node range.

The MPI + CUDA workflow is:

1. MPI divides graph nodes among processes.
2. Each MPI process is intended to use one dedicated GPU.
3. CUDA kernels compute dangling-node contribution, PageRank updates, and convergence difference.
4. MPI collective operations exchange global information between processes.
5. Final PageRank values are gathered and compared with serial output.

The implementation also uses CUDA streams to divide each local node partition into multiple stream segments. This is intended to increase intra-GPU parallelism.

In the available test environment, only one CUDA-capable GPU was available. Therefore, the MPI + CUDA implementation was tested using one MPI process. This validates correctness, but does not fully demonstrate the scalability potential of MPI + CUDA on a multi-GPU system. On a multi-GPU cluster, the expected mapping would be:

MPI Rank 0 $\rightarrow$ GPU 0, MPI Rank 1 $\rightarrow$ GPU 1, MPI Rank 2 $\rightarrow$ GPU 2

Chapter 2

Accuracy Analysis

2.1 RMSE Definition

Root Mean Square Error (RMSE) is used to compare the output of each parallel implementation with the serial implementation.

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (PR_{parallel,i} - PR_{serial,i})^2} \quad (2.1)$$

where:

- $PR_{parallel,i}$ is the PageRank value of node i from a parallel implementation
- $PR_{serial,i}$ is the PageRank value of node i from the serial implementation
- N is the number of nodes

An RMSE value close to zero means that the parallel output is almost identical to the serial output.

2.2 PageRank Sum Validation

PageRank behaves like a probability distribution. Therefore, the sum of all PageRank values should be approximately 1.000000. In the experiments, all tested implementations produced a PageRank sum of approximately 1.000000, confirming that rank mass was preserved during computation.

2.3 Accuracy Results

Implementation	Dataset / Configuration	RMSE vs Serial
MPI	soc-Epinions1, 4 processes	5.99×10^{-20}
MPI	soc-Epinions1, 8 processes	5.13×10^{-20}
CUDA	soc-Epinions1, block size 256	9.21×10^{-20}
CUDA	web-Google, block size 256	2.46×10^{-21}
CUDA	web-BerkStan, block size 256	3.79×10^{-21}
CUDA	WikiTalk, block size 256	1.40×10^{-21}
Hybrid MPI + OpenMP	soc-Epinions1, 2 MPI $\times$ 2 OMP	5.99×10^{-20}
Hybrid MPI + CUDA	soc-Epinions1, 1 MPI + CUDA	4.31×10^{-20}
Hybrid MPI + CUDA	web-Google, 1 MPI + CUDA	1.31×10^{-21}
Hybrid MPI + CUDA	web-BerkStan, 1 MPI + CUDA	5.56×10^{-21}
Hybrid MPI + CUDA	WikiTalk, 1 MPI + CUDA	4.73×10^{-21}

Table 2.1: Accuracy comparison with serial PageRank output

2.4 Accuracy Discussion

The RMSE values are extremely close to zero for all tested implementations. This confirms that the parallel and hybrid versions produce almost the same PageRank values as the serial implementation. Small non-zero RMSE values are expected because of floating-point arithmetic differences and different execution orders in parallel computation.

Chapter 3

Performance and Scalability Analysis

3.1 Experimental Environment

The experiments were performed on a laptop environment using WSL/Linux. The detected CPU configuration inside WSL was:

- 4 logical CPUs visible to WSL
- 2 physical cores visible to WSL
- Intel Core i7-13620H processor

The CUDA tests were performed using:

- NVIDIA GeForce RTX 2050 GPU
- 4 GB GPU memory
- CUDA compiler version 11.5

Because only one CUDA-capable GPU was available, MPI + CUDA was tested using one MPI process. Multi-GPU scaling could not be fully evaluated on the available laptop.

3.2 Speedup and Efficiency Metrics

Speedup is calculated as:

$$Speedup = \frac{T_{serial}}{T_{parallel}} \quad (3.1)$$

Efficiency is calculated as:

$$Efficiency = \frac{Speedup}{Number\ of\ Workers} \times 100\% \quad (3.2)$$

For MPI, the number of workers is the number of MPI processes. For OpenMP and Pthreads, it is the number of threads. For Hybrid MPI + OpenMP, it is:

$$Total\ Workers = MPI\ Processes \times OpenMP\ Threads \quad (3.3)$$

3.3 MPI Performance Results

Processes	Serial Time (s)	MPI Time (s)	Speedup	Efficiency
4	0.108764	0.068599	1.59x	39.6%
8	0.110082	0.113812	0.97x	12.1%

Table 3.1: MPI performance on soc-Epinions1 dataset

The MPI implementation achieved a speedup of 1.59x with 4 processes. However, when 8 processes were used, the performance decreased. This is because the WSL environment provided only 4 logical CPUs, so using 8 processes caused oversubscription. In addition, MPI communication overhead increases as the number of processes increases.

3.4 CUDA Block Size Experiment

CUDA block sizes of 128, 256, and 512 were tested on the soc-Epinions1 dataset.

Block Size	Serial Time (s)	CUDA Time (s)	Speedup	RMSE
128	0.116098	12.160784	0.01x	6.79×10^{-20}
256	0.110481	11.822072	0.01x	9.21×10^{-20}
512	0.111947	12.997754	0.01x	4.81×10^{-20}

Table 3.2: CUDA block size comparison on soc-Epinions1

Block size 256 gave the best time among the tested block sizes. However, CUDA was still slower than serial for this dataset due to GPU overhead, atomic operations, and irregular memory access.

3.5 CUDA Dataset Performance

Dataset	Nodes	Edges	Serial Time (s)	CUDA Time (s)	Speedup
soc-Epinions1	75,888	508,837	0.110481	11.822072	0.01x
web-Google	916,428	5,105,039	0.910194	0.843098	1.08x
web-BerkStan	685,231	7,600,595	0.468605	2.279945	0.21x
WikiTalk	2,394,385	5,021,410	0.729310	0.719082	1.01x

Table 3.3: CUDA performance across datasets using block size 256

CUDA performed better than serial on the web-Google dataset and slightly better on the WikiTalk dataset. However, it was slower on soc-Epinions1 and web-BerkStan. This shows that GPU performance depends not only on dataset size but also on graph structure, edge distribution, memory access pattern, atomic operations, and CPU-GPU transfer overhead.

3.6 Hybrid MPI + OpenMP Performance

MPI Proc.	OMP Threads/Proc.	Total Workers	Serial Time (s)	Hybrid Time (s)	S
2	2	4	0.046975	0.041614	
2	8	16	0.046202	0.048351	

Table 3.4: Hybrid MPI + OpenMP performance on soc-Epinions1

The 2 MPI processes $\times$ 2 OpenMP threads configuration achieved a speedup of 1.13x. However, using 8 OpenMP threads per process created 16 total workers, which exceeded the available logical CPUs in WSL. This caused oversubscription and reduced efficiency.

3.7 Hybrid MPI + CUDA Performance

Dataset	Nodes	Edges	Serial Time (s)	Hybrid Time (s)	Speedup
Sample Graph	4	6	0.000001	0.212278	0.00x
soc-Epinions1	75,888	508,837	0.051095	0.310191	0.16x
web-Google	916,428	5,105,039	1.100793	1.123873	0.98x
web-BerkStan	685,231	7,600,595	0.482861	2.463009	0.20x
WikiTalk	2,394,385	5,021,410	0.674896	1.033161	0.65x

Table 3.5: Hybrid MPI + CUDA performance using 1 MPI process, 4 CUDA streams, and block size 256

The MPI + CUDA implementation produced correct results for all datasets. However, it did not achieve high speedup on the available laptop because only one CUDA-capable GPU was available. Therefore, the implementation was executed with one MPI process. With one MPI process, MPI communication does not happen across multiple nodes, but MPI function overhead, GPU-host data transfer, and stream synchronization overhead still exist.

The implementation is designed for a multi-GPU system where each MPI process uses a separate GPU. In such an environment, MPI + CUDA can distribute the graph across multiple GPUs and potentially achieve better scalability.

3.8 Performance Discussion

The results show that parallel performance depends on the relationship between computation workload and overhead.

For small datasets, parallel implementations can be slower because the overhead of creating threads, launching kernels, transferring data, and communicating between processes is larger than the computation itself. This is clearly visible in the sample graph and soc-Epinions1 CUDA results.

For larger datasets such as web-Google and WikiTalk, CUDA begins to show performance benefits because there is more parallel work available for the GPU. However,

graph algorithms such as PageRank have irregular memory access patterns, and some nodes may have many more incoming edges than others. This causes load imbalance among threads and reduces the achievable speedup.

MPI performance is affected by communication overhead. In PageRank, updated rank values must be shared after each iteration. Therefore, operations such as `MPI_Allgatherv` can become expensive, especially for large graphs.

Hybrid MPI + OpenMP performs well when the number of total workers matches the available CPU resources. However, oversubscription reduces efficiency. Hybrid MPI + CUDA is conceptually strong for multi-GPU systems, but cannot show full potential on a single-GPU laptop.

Chapter 4

User Interface Demonstration

4.1 Flask Dashboard Overview

A simple web-based dashboard was developed using HTML, CSS, JavaScript, and Python Flask. The UI is not part of the main HPC requirement, but it improves demonstration and testing.

The dashboard allows the user to:

- Select a dataset
- Select an implementation
- Enter a parameter such as thread count, process count, or CUDA block size
- Run PageRank from the browser
- View top 10 influential nodes
- View execution time, speedup, efficiency, RMSE, and PageRank sum
- View raw terminal output

4.2 UI Implementation

The Flask backend executes the compiled PageRank binaries using Python's `subprocess` module. The frontend sends the selected dataset, implementation, and parameter to the backend through an API request. The backend runs the selected executable and parses the terminal output using regular expressions.

The UI supports the following implementations:

- Serial
- OpenMP
- Pthreads
- MPI
- CUDA

- Hybrid MPI + OpenMP
- Hybrid MPI + CUDA

The UI is especially useful for demonstration because it presents performance values in a readable format while still showing the raw terminal output for verification.

Chapter 5

Conclusion

This project successfully implemented and analyzed the PageRank algorithm using several HPC programming models. The serial implementation served as the baseline for both correctness and performance comparison. OpenMP and Pthreads demonstrated shared-memory parallelism on CPU cores. MPI demonstrated distributed-memory parallelism using multiple processes. CUDA demonstrated GPU acceleration. Two hybrid methods, MPI + OpenMP and MPI + CUDA, were also implemented to combine multiple parallel programming models.

The accuracy analysis showed that all tested parallel and hybrid implementations produced results very close to the serial implementation. RMSE values were close to zero and the total PageRank sum remained approximately 1.000000, confirming correctness.

The performance analysis showed that parallel speedup depends on dataset size, graph structure, hardware resources, and communication overhead. MPI achieved speedup with 4 processes but degraded with oversubscription. CUDA achieved speedup for some larger datasets, especially web-Google, but was slower for smaller or irregular datasets. Hybrid MPI + OpenMP achieved speedup when the number of workers matched the available CPU resources. Hybrid MPI + CUDA produced correct results but was limited by the single-GPU testing environment.

Overall, the project demonstrates how the same graph algorithm can be mapped to different HPC paradigms and how performance depends not only on parallelism but also on communication, synchronization, memory access, and hardware availability.

Bibliography

- [1] L. Page, S. Brin, R. Motwani, and T. Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*, Stanford InfoLab, 1999.
- [2] Stanford Network Analysis Project, *SNAP Datasets*, <https://snap.stanford.edu/data/>
- [3] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, <https://www.openmp.org/>
- [4] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, <https://www.mpi-forum.org/>
- [5] NVIDIA, *CUDA C++ Programming Guide*, <https://docs.nvidia.com/cuda/>